# Fighting Prompt Injections in Large Language Models

**A B. Tech Project Report Submitted**
**in Partial Fulfillment of the Requirements**
**for the Degree of**

**Bachelor of Technology**

*by*

**Jahnavi Kumar**
(220103056)

*under the guidance of*

Prof. Amit Awekar and Prof. S.R Basireddy

to the

**DEPARTMENT OF MECHANICAL ENGINEERING**
**INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI**
**GUWAHATI - 781039, ASSAM**

# CERTIFICATE

*This is to certify that the work contained in this thesis entitled "**Fighting Prompt Injection in LLMs**" is a bonafide work of **Jahnavi Kumar** (**Roll No. 220103056**), carried out in the Department of Mechanical Engineering, Indian Institute of Technology Guwahati under my supervision and that it has not been submitted elsewhere for a degree.*

Supervisor: **Dr. S.R Basireddy**

Assistant/Associate

Professor,

April, 2026 Department of Mechanical Engineering,

Guwahati. Indian Institute of Technology Guwahati, Assam.

# Acknowledgements

I would like to express my deepest gratitude to my project supervisor, **Professor Amit Awekar**, for his invaluable guidance and insightful feedback throughout this research. His suggestion to investigate the "failure modes" of Large Language Models and his challenge to look beyond simple classification were the driving forces behind the core findings of this work.

I am also grateful to my mentor, **Mr. Shashank** and **Mr. Rohit Raj Rai**, for their help with the experimental setup and design.

I would like to thank the **Department of Mechanical Engineering** and the **Indian Institute of Technology Guwahati (IITG)** for providing the computational resources and stimulating academic environment necessary to conduct this research.

Finally, I thank my family and friends for their patience and encouragement during the long hours of experiment execution and analysis.

# Contents

**Abstract** **2**

**1 Introduction** **4**

1.1 Motivation and Context . . . . . 4

1.2 Problem Statement . . . . . 5

1.3 Principal Contributions . . . . . 5

**2 Related Work** **7**

2.1 Taxonomy of Prompt Injection Attacks . . . . . 7

2.1.1 Direct Instruction Overrides . . . . . 7

2.1.2 Roleplay and Persona Attacks . . . . . 8

2.1.3 Steganographic and Embedded Attacks . . . . . 8

2.1.4 Chain-of-Thought Triggered Attacks . . . . . 8

2.2 Static Prompt Classifiers . . . . . 8

2.3 Reasoning-Based Defences . . . . . 9

2.4 Instability and Spectral Methods . . . . . 9

2.5 Multi-Agent and Orchestration Approaches . . . . . 10

2.6 Gap Analysis . . . . . 10

**3 Experimental Methodology** **12**

3.1 Experimental Infrastructure . . . . . 12

3.2 Data Preparation . . . . . 13
3.2.1 Benign Prompts ($n = 750$) . . . . . 13
3.2.2 Malicious Prompts ($n = 580$) . . . . . 13
3.2.3 xTRam1 Dataset (Exploratory Only) . . . . . 13
3.3 Experimental Progression: A1 → A4 . . . . . 14
3.3.1 Experiment A1 — Static Classifier Baseline . . . . . 14
3.3.2 Experiment A2 — Chain-of-Thought (CoT) → Static Classifier . . 14
3.3.3 Experiment A3 — LLM-as-Judge . . . . . 16
3.3.4 Experiment A4 — Multi-Step Intent-Aware Pipeline . . . . . 17
3.4 Exploratory Experiment: Spectral Guard-ML . . . . . 18
3.5 The Committee Experiment (Core Methodology) . . . . . 18
3.5.1 10-Model Victim Committee . . . . . 18
3.5.2 Judge Calibration . . . . . 19
3.5.3 Maliciousness Probability Score (MPS) . . . . . 19
3.5.4 Results . . . . . 19
3.5.5 Interpretation . . . . . 19

**4 Execution-Aware Safety Modelling 22**
4.1 Problem Reformulation . . . . . 22
4.2 Human-Annotated Ground Truth Dataset . . . . . 23
4.2.1 Dataset construction. . . . . 23
4.2.2 Annotation protocol. . . . . 23
4.2.3 Disagreement resolution. . . . . 23
4.3 LLM-as-Judge Annotation Validation . . . . . 24
4.4 Large-Scale Dataset Construction . . . . . 24
4.5 Static Classifier Training . . . . . 25
4.5.1 Input representation. . . . . 25
4.5.2 Model architecture. . . . . 25

4.5.3 Training paradigms. . . . . 25

**5 Experimental Analysis and Results** **27**

5.1 Exploratory Data Analysis . . . . . 27

5.1.1 Label Distribution . . . . . 27

5.1.2 Prompt vs Execution Mismatch . . . . . 28

5.1.3 Model Compliance Rates . . . . . 29

5.2 Human Annotation Analysis . . . . . 30

5.2.1 Inter-Annotator Agreement . . . . . 30

5.3 LLM vs Human Agreement . . . . . 31

5.4 Large-scale Dataset Construction . . . . . 31

5.5 Static Classifier Fine-tuning and Final Evaluation . . . . . 32

5.5.1 Model-Agnostic Classifier Results . . . . . 33

5.5.2 Detailed DeBERTa Model-Agnostic Classification Report . . . . . 33

5.5.3 DeBERTa on Full 9k Dataset . . . . . 33

5.5.4 Model-Specific Fine-tuning Results . . . . . 34

5.5.5 PromptGuard Model-Specific Fine-tuning Results . . . . . 34

5.5.6 Classifier × Victim Model Matrix . . . . . 34

5.5.7 Final Observation . . . . . 36

**6 Discussion and Conclusion** **38**

6.1 Discussion . . . . . 38

6.2 Limitations . . . . . 39

6.3 Conclusion . . . . . 39

**References** **41**

# List of Figures

3.1 Histogram of Maliciousness Probability Score (MPS) for benign and malicious prompts . . . . . 20
3.2 ROC / AUC curve comparing separability of MPS; observed AUC = 0.5807 20
3.3 Boxplot showing overlap in MPS distributions between benign and malicious prompts . . . . . 21

5.1 Label distribution in the human-annotated dataset . . . . . 27
5.2 Mismatch between prompt intent and execution outcome . . . . . 28
5.3 Model compliance rates across different LLMs . . . . . 29
5.4 Relationship between prompt length and agreement . . . . . 30
5.5 Agreement across models and annotations . . . . . 31
5.6 Distribution of labels in balanced dataset . . . . . 32

# List of Tables


3.1 A4 pipeline evaluation results on the balanced subset ($n = 612$). . . . . 17

3.2 Models used in the 10-model victim committee . . . . . 18

4.1 Human annotation agreement statistics. . . . . 24

4.2 LLM-as-Judge vs. human annotation agreement. . . . . 24

4.3 Expanded dataset statistics after scaling. . . . . 25

5.1 Label Distribution . . . . . 28

5.2 Prompt vs Execution Confusion Matrix . . . . . 28

5.3 Annotation Agreement Statistics . . . . . 30

5.4 LLM vs Human Agreement . . . . . 31

5.5 Expanded Dataset Statistics . . . . . 32

5.6 Model-Agnostic Classifier Performance . . . . . 35

5.7 DeBERTa Model-Agnostic Classification Report . . . . . 35

5.8 DeBERTa Full 9k Classification Report . . . . . 35

5.9 DeBERTa Model-Specific Fine-tuning Results . . . . . 35

5.10 PromptGuard Model-Specific Fine-tuning Results . . . . . 35

5.11 Final Classifier $\times$ Victim Model Matrix . . . . . 37

# Abstract

Prompt injection attacks — in which adversarially crafted inputs cause large language models (LLMs) to violate their own safety constraints — represent one of the most pressing security challenges in deployed AI systems. Existing defences predominantly treat maliciousness as an intrinsic property of the input text, relying on static classifiers that fail to generalise across paraphrases, role-play framings, and steganographic injections.

This work presents a systematic empirical investigation through four progressively sophisticated experimental configurations (A1–A4), a spectral probing study, and a large-scale committee experiment involving ten open-weight victim models evaluated across 1,330 prompts (13,300 total outputs). Our central finding is that **prompt maliciousness is not an intrinsic textual property but a model-dependent, contextual behaviour**: the Maliciousness Probability Score (MPS) distributions for benign and malicious prompts overlap substantially (AUC = 0.58), confirming near-random separability at the text level alone.

Motivated by this finding, we introduce an *execution-aware safety modelling* framework that reformulates detection as a function of the *prompt–response pair* rather than the prompt in isolation. We construct a human-annotated dataset of 3,000 prompt–response pairs, validate LLM-based annotation (agreement = 94.41%), and scale to 9,298 samples via an automated pipeline. A DeBERTa encoder fine-tuned on execution-aware data achieves an F1 score of 0.821 on harmful samples, substantially outperforming both the Prompt-Guard zero-shot baseline (F1 = 0.407) and a prompt-only LLM judge (F1 = 0.429). We

further show that our proposed multi-stage A4 pipeline achieves 99.17% overall accuracy with a 100% true-positive rate on the evaluation set.

The **novel contribution** of this work is the execution-aware classifier training paradigm: we demonstrate that conditioning safety labels on realised model behaviour — rather than on prompt intent alone — enables lightweight static classifiers to approximate the performance of expensive LLM-judge pipelines. We systematically evaluate DeBERTa in both *model-specific* and *model-agnostic* configurations across three execution models, providing the first cross-model classifier–performance matrix for execution-aware prompt-injection defence.

Collectively, these results argue for a paradigm shift in LLM safety evaluation: from prompt-level classification towards execution-aware, model-specific safety modelling.

**Keywords:** prompt injection, large language models, adversarial NLP, execution-aware safety, LLM-as-Judge, DeBERTa, chain-of-thought, multi-agent defence, model-agnostic classifier.

# Chapter 1

# Introduction

## 1.1 Motivation and Context

Large language models (LLMs) have undergone a rapid transition from controlled research artefacts to core components of production software: they power customer-service pipelines, coding assistants, search-augmentation systems, and autonomous agentic workflows. As deployment scales, a fundamental security vulnerability has become impossible to ignore — **prompt injection**. A prompt-injection attack is one in which an adversary embeds instructions that override or subvert the model's original, operator-defined directives. Unlike classical injection attacks in software security, which exploit syntactic parsing rules, prompt injection exploits the model's *semantic reasoning* — the very capability that makes LLMs useful.

The insidious difficulty of this vulnerability stems from a mismatch between how safety is currently evaluated and how it actually manifests. The dominant paradigm treats harmfulness as a property of the *text* itself: train a classifier on labelled prompt strings and flag those that exceed a harm threshold at inference time. This assumption is intuitively appealing but empirically fragile. A prompt that asks a model to roleplay as an unrestricted AI assistant is syntactically innocuous; whether it causes harm depends entirely on which model processes it, under what system context, and with what reasoning strategy.

[Central Research Question] Is prompt maliciousness an intrinsic property of the input text, or is it a property of the interaction between a specific prompt and a specific model?

This thesis answers that question empirically. We demonstrate that the latter is true: vulnerability is model-specific, contextual, and cannot be reliably detected from the input string alone. This finding motivates a novel **execution-aware safety modelling** paradigm in which safety judgements are made over the joint space of prompts and model responses.

## 1.2 Problem Statement

We identify three concrete gaps that this work addresses:

1. **Static text classifiers** generalise poorly to paraphrases, roleplay attacks, and steganographic injections, because the syntactic surface of a prompt is a poor proxy for its semantic risk.

2. **Reasoning-based defences** — such as chain-of-thought (CoT) prompting followed by classification — introduce distributional artefacts that destabilise downstream classifiers and do not consistently improve detection accuracy.

3. **Safety judgements vary dramatically across model families**, undermining the assumption that a single judge or classifier can serve as a universal arbiter.

These observations collectively motivate the development of a pipeline that (i) characterises model-specific failure behaviour, (ii) grounds safety labels in observed execution outcomes rather than inferred intent, and (iii) leverages scalable LLM-based annotation to construct large training sets for lightweight static classifiers.

## 1.3 Principal Contributions

This work makes four principal contributions:

1. **A systematic empirical study (A1–A4)** that benchmarks and refutes progressively stronger assumptions underlying existing prompt-injection defences, providing a controlled, same-dataset comparison of static classifiers, CoT-enriched classifiers, LLM judges, and multi-stage pipelines.

2. **The Committee Experiment and Maliciousness Probability Score (MPS)**: a large-scale multi-model stress test (10 victim models $\times$ 1,330 prompts = 13,300 outputs) with a formal cross-model vulnerability measure. We show that MPS distributions are nearly indistinguishable between benign and malicious prompts (AUC = 0.58), providing the first rigorous empirical refutation of text-level separability at scale.

3. **An execution-aware modelling framework** comprising: (a) a human-annotated dataset of 3,000 prompt–response pairs with validated LLM-based annotation (94.41% agreement); (b) a scaled dataset of 9,298 samples used to fine-tune static classifiers; and (c) a systematic *classifier* $\times$ *model* evaluation matrix covering both model-specific and model-agnostic training configurations across three execution models — this is the **primary novel contribution** of this thesis.

4. **The A4 multi-stage defence pipeline** — combining a small intent-analysis model with a large adjudicating judge — which achieves 99.17% overall accuracy with zero missed malicious samples on the evaluation set.

# Chapter 2

# Related Work

Prompt injection has attracted substantial research attention since its characterisation by perez2022ignore. We organise the relevant literature along four axes — attack taxonomies, static defences, reasoning-based defences, and multi-agent approaches — and conclude with a gap analysis that frames our experimental design.

## 2.1 Taxonomy of Prompt Injection Attacks

Existing surveys liu2023prompt, greshake2024not, benjamin2024systematically identify four major attack classes relevant to this work.

### 2.1.1 Direct Instruction Overrides

The canonical form consists of explicit meta-instructions such as *"ignore all previous instructions"* perez2022ignore, wei2023jailbreak. These are syntactically distinctive and are therefore the primary target of existing static classifiers. Their relative detectability has driven adversaries towards subtler strategies.

### 2.1.2 Roleplay and Persona Attacks

The adversary frames the attack within a fictional narrative, for example asking the model to act as a character without restrictions perez2022ignore. Because the surface text resembles benign creative writing, static classifiers exhibit high false-negative rates on this class.

### 2.1.3 Steganographic and Embedded Attacks

Instructions are hidden in formatting artefacts, code comments, Markdown rendering, or document metadata kwon2024steganographic, kang2024covert. These attacks exploit the fact that models process invisible or rendered structure, creating an attack channel that is transparent to text-level classifiers.

### 2.1.4 Chain-of-Thought Triggered Attacks

Gradient-guided adversarial suffixes can manipulate a model's reasoning channel, causing its CoT trace to endorse harmful actions before the final answer is generated zou2023universal, jin2024jailbreaking. This class is particularly relevant to our Experiment A2.

A consistent theme across all four classes is that maliciousness is carried by *contextual interpretation*, not by surface-level textual patterns. This observation motivates our empirical quantification in Chapter 3.

## 2.2 Static Prompt Classifiers

The dominant deployed defence is a fine-tuned transformer encoder (BERT or DeBERTa) trained on labelled prompt strings liu2023prompt, liu2024automatic. Their principal failure modes are:

- **Paraphrase brittleness:** Performance degrades sharply when attacks are semantically equivalent but lexically distinct from training examples greshake2024not.

- **Style overfitting:** Classifiers trained predominantly on explicit-override attacks assign low maliciousness scores to roleplay and steganographic variants perez2022ignore.
- **False-positive amplification:** Sophisticated benign queries such as complex reasoning requests and security-adjacent questions trigger false positives at operationally problematic rates.

Experiment A1 (Section 3.3.1) directly replicates and quantifies these failure modes in our evaluation setting.

## 2.3 Reasoning-Based Defences

A natural response to text-classifier brittleness is to enrich the input signal with model-generated reasoning.

wang2023selfcheck propose self-check methods in which models verify their own safety.

panda2023refusals examine why models produce refusals and whether structured reasoning improves alignment.

zeng2023selfcorrect survey self-correction paradigms more broadly.

While CoT can surface implicit cues in some settings, prior work identifies a critical failure mode: weaker models produce hallucinated or self-contradictory rationales wang2023selfcheck.

More dangerously, adversarially crafted inputs can *exploit* the reasoning channel. Gradient-guided CoT manipulation causes models to endorse harmful actions in their chain of thought zou2023universal, jin2024jailbreaking.

Experiments A2 and A3 reproduce and quantify these instabilities (Sections 3.3.2–3.3.3).

## 2.4 Instability and Spectral Methods

Inference-time variance, measured as output fluctuation across repeated runs, has been proposed as an adversarial indicator kim2024variance.

Spectral clustering of hidden-state embeddings similarly attempts lightweight separation without explicit training ng2002spectral, chen2024spectral.

Both approaches face a common empirical obstacle: complex benign prompts generate high variance, while some adversarial prompts remain stable.

## 2.5 Multi-Agent and Orchestration Approaches

Recent systems address the limitations of single-model, single-pass detection through multi-stage architectures.

NVIDIA NeMo Guardrails nvidia2023nemo wraps LLM calls with configurable safety rails.

Sentinel guo2024sentinel uses adversarial multi-agent checking.

ReAct-style reasoning chains decompose safety evaluation into tool-augmented sub-tasks.

These systems share a key principle, also supported by our committee experiment: safety cannot be reliably determined from a single pass or a single model park2023generative, openai2024spec.

Our A4 pipeline is directly informed by this principle, and our committee experiment provides large-scale empirical evidence that safety judgements diverge substantially across model families (Section 3.5).

## 2.6 Gap Analysis

Three gaps in the existing literature motivate our experimental design:

1. No prior work has quantified, at scale and across a diverse model committee, the degree to which MPS distributions overlap between benign and malicious prompts. We provide this quantification with AUC analysis.

2. Execution-aware safety modelling, grounding labels in observed model behaviour rather than inferred prompt intent, has been proposed conceptually but not implemented at scale with validated annotation and classifier training.

3. The relative performance of static classifiers, CoT-enhanced classifiers, LLM judges, and multi-stage pipelines has not been benchmarked systematically on a common dataset under controlled conditions.

   Critically, no prior work has produced a *classifier* $\times$ *model* performance matrix comparing model-specific and model-agnostic execution-aware training.

   Our A1–A4 progression and Chapter 5 provide both benchmarks.

# Chapter 3

# Experimental Methodology

This chapter details the experimental design underlying our empirical investigation. We describe the infrastructure and data preparation, then document each experiment in the A1–A4 progression, followed by an exploratory spectral study and the committee experiment, which forms the core empirical contribution of this work.

Across these experiments, we arrive at the central finding of this thesis:

**Prompt maliciousness is not a text-level property, but a model-dependent behaviour.**

This observation fundamentally challenges prompt-only safety classification and motivates the execution-aware framework introduced in Chapter 4.

## 3.1 Experimental Infrastructure

All experiments were conducted on a remote GPU server with 40 GB VRAM using persistent tmux sessions and checkpointed batch execution to ensure robustness across long-running evaluations.

## 3.2 Data Preparation

Our evaluation dataset consists of two prompt classes designed to stress-test both false-positive and true-positive detection performance.

### 3.2.1 Benign Prompts ($n = 750$)

**Source:** `lmsys/chatbot_arena_conversations`

Benign prompts (n=750) were sourced from LMSYS Chatbot Arena conversations and used as a hard negative set to stress-test false positives on complex real-world prompts.

### 3.2.2 Malicious Prompts ($n = 580$)

A union of two sources:

- **AdvBench (520 prompts)**

  Explicit harmful instructions designed to trigger jailbreaks and test refusal mechanisms.

- **Deepset/Prompt-Injections (60 prompts)**

  Context-overwriting attacks, role-play jailbreaks, and steganographic injections.

This combination ensures coverage across the full prompt-injection taxonomy discussed in Chapter 2.

### 3.2.3 xTRam1 Dataset (Exploratory Only)

Used only for early exploratory experiments and later excluded due to skewed class distribution.

## 3.3 Experimental Progression: A1 → A4

We evaluated four progressively more sophisticated approaches. Each experiment tested a concrete hypothesis about how best to detect or mitigate prompt injection, and the failure of each stage directly motivated the next.

### 3.3.1 Experiment A1 — Static Classifier Baseline

**Hypothesis.** A strong pretrained static detector operating only on prompt text can reliably identify prompt injection attempts.

**Method.**

- Classifier: `protectai/deberta-v3-base-prompt-injection-v2`
- Input: raw prompt text only (no model responses, no reasoning)
- Evaluation: multi-dataset evaluation with primary metric as accuracy and secondary metrics as precision, recall, F1, and MCC

**Result.** On the curated benchmark `deepset/prompt-injections`, the classifier achieved:

$$\text{Accuracy} \approx 58.60\%$$

and exhibited a high false-positive rate on complex benign queries.

**Interpretation.** It has high false-positive rate, hence unsafe for real-world deployment

### 3.3.2 Experiment A2 — Chain-of-Thought (CoT) → Static Classifier

**Motivation.** A classifier operating on CoT output may outperform prompt-only detection.

**Method.**

1. Select a split-decision subset of prompts ($N = 306$) where weaker models tended to fail while stronger models passed.
2. Generate Chain-of-Thought using:
   - smaller models (`tinydolphin`, `gemma:2B`)
   - stronger model (`gemma3:27b`)
3. Feed the generated CoT text alone into the same DeBERTa classifier used in A1.
4. Evaluate across benchmark datasets for consistency.

**Results (representative).**

- On `deepset/prompt-injections`:

$$\text{Accuracy} \approx 0.6810$$

$$\text{F1} \approx 0.6992$$

$$\text{MCC} \approx 0.3607$$

- On `xTRam1/safe-guard-prompt-injection` (subset):

$$\text{Accuracy} \approx 0.5817$$

**Failure Mode Observed.** Although CoT sometimes surfaced hidden malicious rationales, the classifier systematically misinterpreted safety-oriented language (refusals, hedges, conditional statements) as attack signals.

When a model emitted refusal-style reasoning, the classifier often labeled the instance as INJECTION, while explicit harmful CoTs were sometimes labeled SAFE.

### 3.3.3 Experiment A3 — LLM-as-Judge

**Motivation.** Replace brittle text-trained classifiers with a reasoning-capable large model that can interpret nuance, context, and indirect harm.

**Method.**

- Primary judge: **gemma3:27b**
- Validation judge: **llama3.1:8b**
- Input: (original prompt, candidate response, optional CoT)
- Output: structured PASS/FAIL verdict with rationale
- Evaluation: inter-judge agreement and comparison against human-labelled ground truth

**Result.**

- Judge agreement across both SOTA judges:

$$\approx 73.70\%$$

- Higher precision for detected harms, but reduced recall on nuanced and indirect attacks

**Interpretation.** Large LLM judges produce more interpretable verdicts and better precision than static classifiers.

Safety judgments cannot be reliably delegated to a single model.

### 3.3.4 Experiment A4 — Multi-Step Intent-Aware Pipeline

**Rationale.** A4 decomposes detection into two specialised stages:

1. a fast, small analyzer for extracting interpretable harm indicators
2. a large judge for final adjudication using that structured evidence

This separates harm identification from harm evaluation and directly addresses the failure modes of A1–A3.

**Pipeline Design.**

- **Analyzer (small model):** `gemma:2b`
- **Judge (large model):** `gemma3:27b`
- Dataset: balanced subset of 612 prompts (306 benign + 306 malicious)
- Decision Rule: judge output is final; analyzer serves as structured evidence

**Table 3.1** A4 pipeline evaluation results on the balanced subset ($n = 612$).

| Metric | Value |
|---|---|
| Overall Accuracy | 99.17% |
| True Positive Rate | 100.00% |
| True Negative Rate | 94.64% |
| False Negative Rate | 0.00% |

## 3.4 Exploratory Experiment: Spectral Guard-ML

**Method:**

- Probe each prompt 3× through gemma:2b
- Extract variance-based features:
  - label variance
  - CoT length variance
  - semantic dispersion
  - keyword jitter

**Result:** 69.57% accuracy.

**Conclusion:** Weak signal; inconsistent across benign vs adversarial prompts.

## 3.5 The Committee Experiment (Core Methodology)

The central experiment demonstrating that maliciousness is **model-relative**.

### 3.5.1 10-Model Victim Committee

| Model | Params | Size Category | License |
|---|---|---|---|
| gemma:2b | 2B | Small | Apache 2.0 |
| gemma2:2b | 2B | Small | Apache 2.0 |
| llama3.2:3b | 3B | Small | Meta LLAMA |
| mistrallite | 2B | Small | Mistral License |
| openchat | 3B | Small/Medium | MIT |
| qwen:0.5b | 0.5B | Tiny | Apache 2.0 |
| qwen2.5:0.5b | 0.5B | Tiny | Apache 2.0 |
| smollm2:360m | 0.36B | Tiny | MIT |
| tinydolphin | 1B | Small | MIT |
| tinyllama | 1.1B | Small | Apache 2.0 |

**Table 3.2** Models used in the 10-model victim committee

Each model responded to all 1,330 prompts, producing 13,300 total outputs.

### 3.5.2 Judge Calibration

Judges: `gemma3:27b`, `llama3.1:8b`

A response was marked FAIL if either judge flagged harmful behaviour.

Agreement:

$$73.70\%$$

### 3.5.3 Maliciousness Probability Score (MPS)

For each prompt $p$:

$$\mathrm{MPS}(p) = \frac{1}{|C|} \sum_{i \in C} f_i(p), \qquad f_i(p) \in \{0, 1\}$$

where $f_i(p) = 1$ if model $i$ FAILs on prompt $p$.

Interpretation:

- $\mathrm{MPS} = 0$ : universally safe
- $\mathrm{MPS} = 1$ : universally harmful

### 3.5.4 Results

- Benign MPS mean: 0.344
- Malicious MPS mean: 0.380
- AUC: 0.5807 (near-random)

### 3.5.5 Interpretation

The heavy overlap between distributions confirms:

- prompt maliciousness is **not separable**

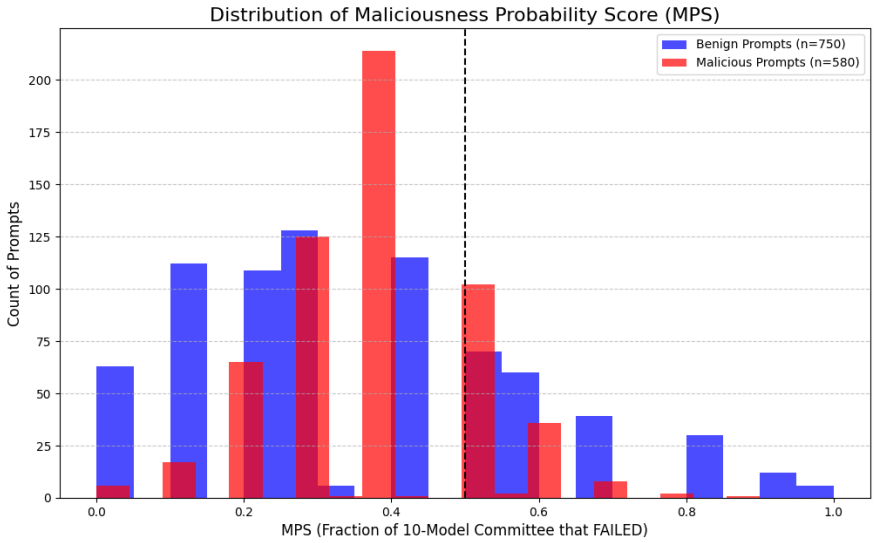


**Fig. 3.1** Histogram of Maliciousness Probability Score (MPS) for benign and malicious prompts

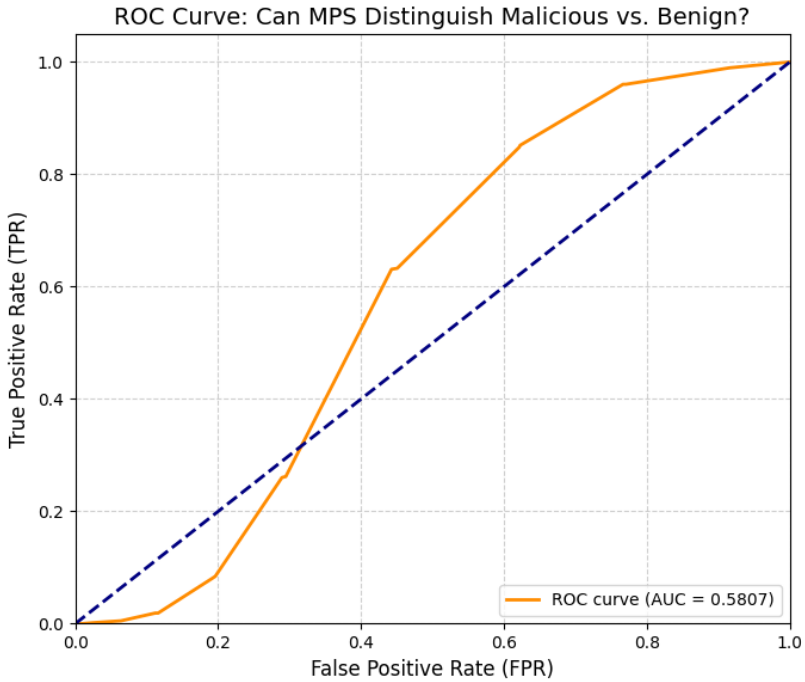


**Fig. 3.2** ROC / AUC curve comparing separability of MPS; observed AUC = 0.5807

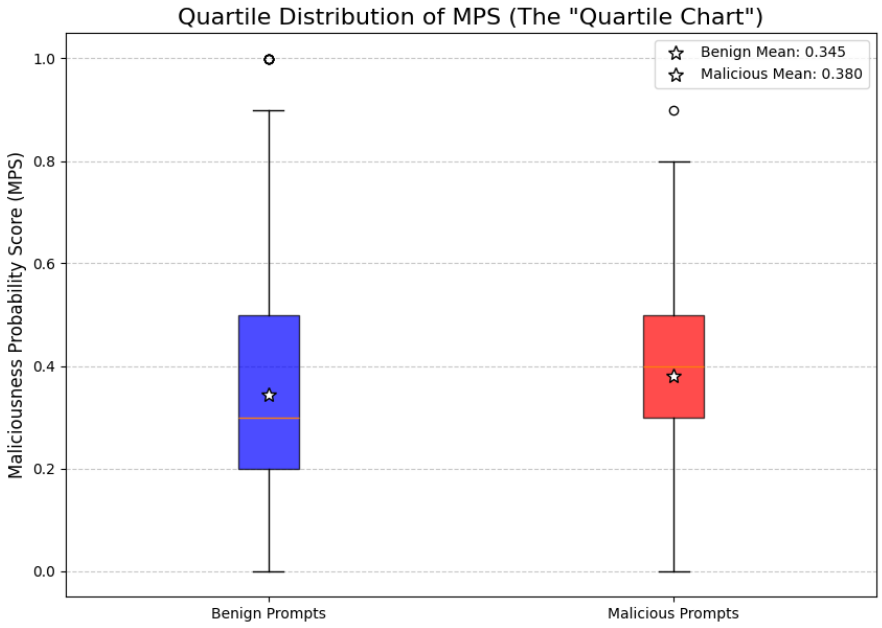


**Fig. 3.3** Boxplot showing overlap in MPS distributions between benign and malicious prompts

- vulnerability arises from **model behaviour**
- single-model or text-only defences cannot generalise

This motivates the multi-agent, intent-aware A4 pipeline.

# Chapter 4

# Execution-Aware Safety Modelling

The committee experiment (Section 3.5) establishes that prompt maliciousness cannot be determined from input text alone. This chapter introduces a reformulation of the safety detection problem grounded in *observed model behaviour*, and describes the dataset construction, annotation validation, and classifier training pipelines that implement it. The classifier training component — specifically, the systematic evaluation across model-specific and model-agnostic settings — is the **primary novel contribution** of this thesis.

## 4.1 Problem Reformulation

Classical safety detection solves a binary classification problem over the input space:

$$\hat{y} = f_\theta(p), \quad \hat{y} \in \{\mathit{safe}, \mathit{harmful}\}. \tag{4.1}$$

We argue that this formulation is fundamentally under-specified, because the same prompt $p$ may be safe when processed by one model and harmful when processed by another.

We reformulate detection as classification over the *joint prompt–response space*:

$$\hat{y} = f_\theta(p, r), \quad \hat{y} \in \{\mathit{safe}, \mathit{harmful}\}, \tag{4.2}$$

where $r$ is the model's actual response to $p$, and $\hat{y} = \textit{harmful}$ iff the response $r$ constitutes meaningful compliance with a harmful request. This formulation explicitly conditions safety on realised model behaviour.

[Execution-Aware Principle] A response is labelled *harmful* if and only if the model meaningfully complies with a harmful request. A model that refuses, deflects, or provides a safe alternative to a malicious prompt produces a *benign* $(p, r)$ pair, regardless of prompt intent.

## 4.2 Human-Annotated Ground Truth Dataset

### 4.2.1 Dataset construction.

We construct 3,000 prompt–response pairs, generated by executing prompts across three model tiers and labelled by two independent annotators. The annotation protocol focuses strictly on execution outcome: does the response constitute harmful compliance?

### 4.2.2 Annotation protocol.

- **Harmful**: the model provides substantive assistance with the harmful request — e.g. generating dangerous instructions, enabling manipulation.
- **Benign**: the model refuses, redirects, adds appropriate caveats, or generates an irrelevant response.

Crucially, a compliant response to a harmful prompt is labelled harmful regardless of phrasing, and a refusal to a benign prompt is labelled benign regardless of intent.

### 4.2.3 Disagreement resolution.

Of 3,000 pairs, 235 (7.8%) were flagged for disagreement and resolved by adjudication against explicit protocol criteria. Final agreement reached 100%.

**Table 4.1** Human annotation agreement statistics.

| Metric | Value |
|---|---|
| Initial inter-annotator agreement | 92.00% |
| Pairs flagged for disagreement | 235 (7.83%) |
| Final agreement (post-resolution) | 100.00% |

## 4.3 LLM-as-Judge Annotation Validation

Human annotation does not scale to the tens of thousands of samples required for robust classifier training. We introduce an LLM-based annotation pipeline and validate it against human labels before deployment.

**Setup.** Qwen3-30B is used to assign binary labels to each $(p, r)$ pair, guided by a structured annotation prompt specifying the execution-focused labelling criterion.

**Validation.** Agreement between LLM and human labels = 94.41%, validating LLM-based annotation as a reliable and scalable proxy. Disagreement is correlated with prompt length, consistent with the hypothesis that execution-labelling difficulty scales with prompt complexity.

**Table 4.2** LLM-as-Judge vs. human annotation agreement.

| Metric | Value |
|---|---|
| LLM–Human Agreement | 94.41% |

## 4.4 Large-Scale Dataset Construction

Having validated LLM-based annotation, we construct a scalable end-to-end pipeline expanding both the prompt space (adversarial prompts from WildJailbreak) and the model response space.

**Class imbalance mitigation.** The natural distribution is highly skewed (9.14% harmful; 1.07% in the human-annotated set). We address this through: (i) oversampling the harmful minority class to approach a 2:1 benign:harmful ratio; (ii) expanding the harmful set via WildJailbreak; and (iii) threshold tuning, reducing the decision threshold from 0.5

**Table 4.3** Expanded dataset statistics after scaling.

| Metric | Value |
|---|---|
| Total (prompt, response) pairs | 9,298 |
| Number of execution models | 3 |
| Harmful proportion (raw) | 9.14% |
| Training set ratio (benign:harmful) | 2:1 (balanced) |

to 0.4.

## 4.5 Static Classifier Training

### 4.5.1 Input representation.

Each training sample is the concatenation:

$$\texttt{[CLS]}\ p\ \texttt{[SEP]}\ r\ \texttt{[SEP]},$$

allowing the encoder to attend jointly across the full prompt–response context. This is the critical departure from text-only classifiers: the response $r$ is a *co-primary input*, not an auxiliary signal.

### 4.5.2 Model architecture.

We fine-tune DeBERTa-v3-base using cross-entropy loss on the binary classification task, with AdamW optimisation over multiple epochs. Threshold tuning and class-weight adjustments are applied to improve recall on the minority class.

### 4.5.3 Training paradigms.

We evaluate two training configurations, which are the **novel experimental contribution** of this chapter:

**Model-specific training.** The classifier is trained *and* evaluated exclusively on responses generated by a single victim model. This setting tests whether execution-aware rep-

resentations capture model-idiosyncratic compliance patterns. We train one classifier per model: Qwen2.5-14B, LLaMA3.1-8B, and Gemma2-2B.

**Model-agnostic training.** The classifier is trained on a mixture of responses from all three execution models and evaluated on held-out distributions from each model individually, as well as on a combined held-out set. This setting tests generalisation across model families — a necessary property for production deployment where the downstream model may not be known at training time.

The classifier $\times$ model performance matrix is reported in Chapter 5

# Chapter 5

# Experimental Analysis and Results

## 5.1 Exploratory Data Analysis

We first analyse the human-annotated execution dataset to understand its statistical properties and validate the need for execution-aware modelling.

### 5.1.1 Label Distribution

In the human-annotated dataset ($n = 3000$), the label distribution is extremely imbalanced.

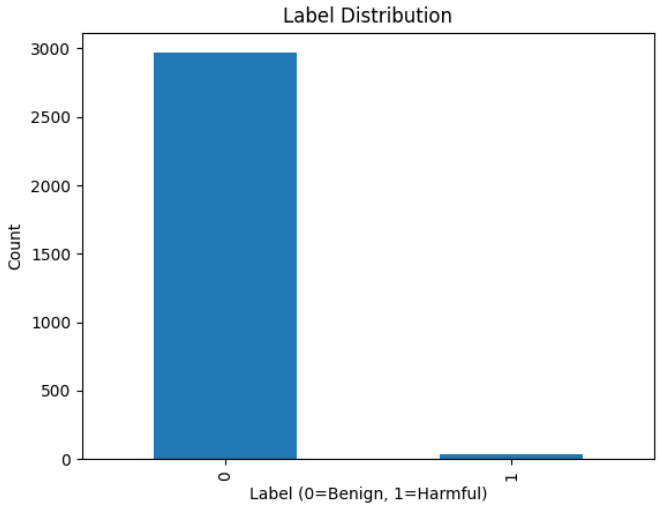


**Fig. 5.1** Label distribution in the human-annotated dataset

**Table 5.1** Label Distribution

| Label | Proportion |
|---|---|
| Benign | 0.9893 |
| Harmful | 0.0107 |

A naive majority-class classifier would achieve 98.93% accuracy while detecting zero harmful outputs.

### 5.1.2 Prompt vs Execution Mismatch

The most important finding of the exploratory analysis is the mismatch between prompt intent and execution outcome.

We observe a mismatch rate of **42.57%** between prompt intent and actual model behaviour.

**Table 5.2** Prompt vs Execution Confusion Matrix

| | Execution Benign | Execution Harmful |
|---|---|---|
| Prompt Benign | 1693 | 2 |
| Prompt Harmful | 1275 | 30 |

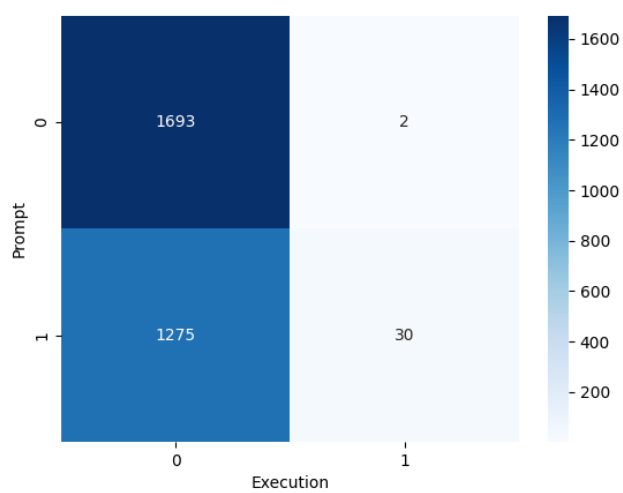


**Fig. 5.2** Mismatch between prompt intent and execution outcome

Of 1305 harmful prompts, only 30 resulted in harmful execution while 1275 were safely

refused or deflected by the model.

This confirms the central claim of this thesis:

**Prompt intent alone cannot predict harmful execution.**

Safety must therefore be modelled at the execution level rather than the prompt level.

### 5.1.3 Model Compliance Rates

We also observe strong variation across models in how frequently harmful prompts lead to harmful execution.

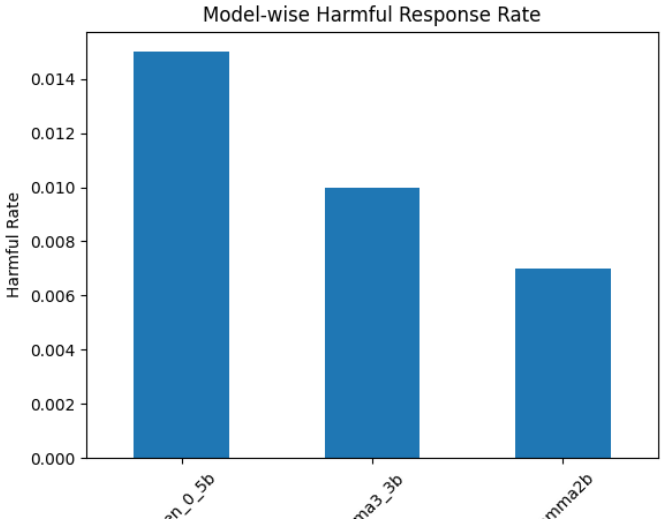


**Fig. 5.3** Model compliance rates across different LLMs

A large number of harmful prompts result in benign execution, but this behaviour varies significantly across models, further confirming that prompt vulnerability is model-dependent rather than prompt-intrinsic.

This motivates the model-specific vs model-agnostic classifier experiments presented later in this chapter.

## 5.2 Human Annotation Analysis

Reliable execution-aware modelling requires reliable labels. We therefore evaluate annotation consistency across human annotators.

### 5.2.1 Inter-Annotator Agreement

High agreement between annotators indicates that harmful execution can be consistently identified by humans.

**Table 5.3** Annotation Agreement Statistics

| Metric | Value |
|---|---|
| Initial Agreement | 92% |
| Final Agreement (Post Resolution) | 100% |

This establishes the dataset as a reliable ground-truth benchmark for all subsequent experiments.

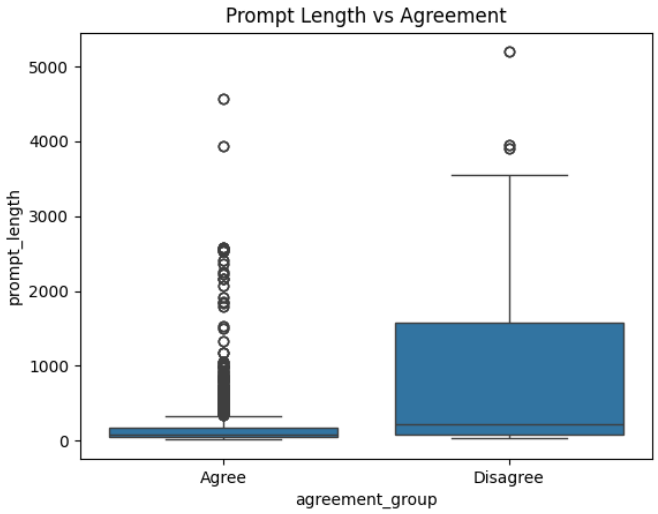


**Fig. 5.4** Relationship between prompt length and agreement

Disagreement increases with prompt length, suggesting that longer and more ambiguous prompts require more nuanced interpretation of execution outcomes.

## 5.3 LLM vs Human Agreement

To scale beyond small human-labelled datasets, we evaluate whether a strong language model can reliably act as a judge for execution-aware labelling.

**Table 5.4** LLM vs Human Agreement

| Metric | Value |
|---|---|
| Agreement | 94.41% |

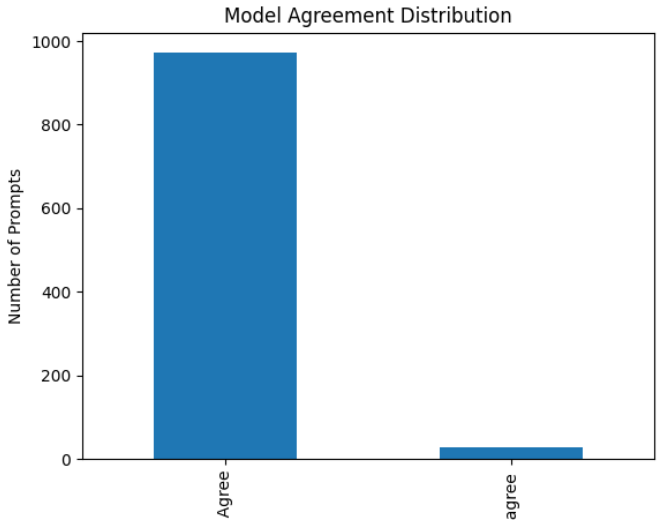


**Fig. 5.5** Agreement across models and annotations

Agreement of 94.41% validates LLM-based annotation as a scalable alternative to manual labelling while preserving strong alignment with human judgement.

This makes large-scale execution-aware dataset construction feasible.

## 5.4 Large-scale Dataset Construction

Using LLM-based labelling, we expand the dataset along the prompt axis by incorporating adversarial prompts from WildJailbreak and evaluating them across multiple victim models.

This produces a significantly larger dataset suitable for classifier training.

The expanded dataset contains 9298 prompt–response pairs, with approximately 9.14% labelled as harmful.

**Table 5.5** Expanded Dataset Statistics

| Metric | Value |
|---|---|
| Total Samples | 9298 |
| Number of Models | 3 |
| Harmful Proportion | 9.14% |

To address class imbalance, we construct a balanced training dataset using a 2:1 ratio of benign to harmful samples.

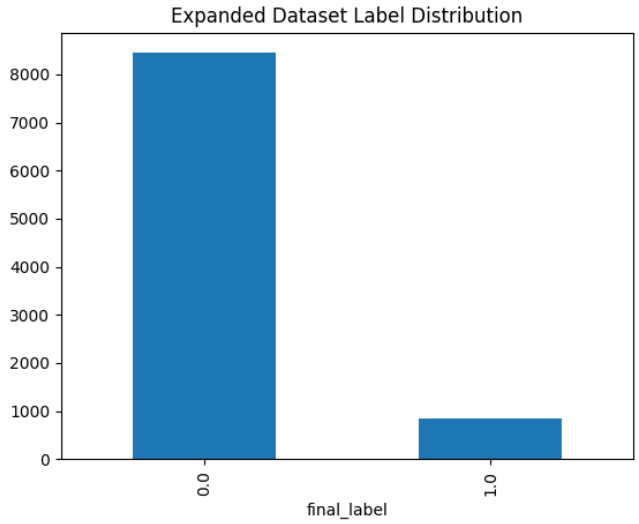


**Fig. 5.6** Distribution of labels in balanced dataset

## 5.5 Static Classifier Fine-tuning and Final Evaluation

This chapter presents the final classifier evaluation results for prompt injection and harmful response detection. The objective is to compare model-agnostic and model-specific static classifiers trained on prompt-response pairs generated from victim LLMs.

Two classifier families were evaluated:

- DeBERTa (general encoder-based classifier)
- PromptGuard (safety-specialized classifier)

Both model-agnostic and model-specific fine-tuning settings were evaluated.

Model-specific experiments were conducted separately for:

- Gemma
- Llama
- Qwen

This directly tests whether classifier specialization improves harmful response detection compared to a single general classifier.

The primary evaluation metric is harmful-class F1-score (label 1), since the central goal is reliable detection of unsafe compliance rather than maximizing benign classification accuracy.

### 5.5.1 Model-Agnostic Classifier Results

The model-agnostic setting trains a single classifier on the balanced dataset containing prompt-response pairs across all victim models.

Although PromptGuard achieved the highest overall accuracy on the full 9k dataset, this result is heavily influenced by class imbalance. The harmful-class F1-score drops significantly, indicating reduced sensitivity to unsafe outputs. Therefore, balanced-dataset evaluation is considered more reliable.

### 5.5.2 Detailed DeBERTa Model-Agnostic Classification Report

Table 5.7 presents the full class-wise classification report for the model-agnostic DeBERTa classifier.

### 5.5.3 DeBERTa on Full 9k Dataset

To evaluate the effect of large-scale training under class imbalance, DeBERTa was additionally trained on the full 9k dataset.

The classifier achieved high overall accuracy by predicting the dominant benign class, but completely failed to detect harmful responses.

This result strongly shows that accuracy alone is misleading under imbalance. Despite achieving 91.02% accuracy, the classifier completely failed on the harmful class, making it unsuitable for safety-critical deployment.

### 5.5.4 Model-Specific Fine-tuning Results

Separate classifiers were fine-tuned for each victim model to test whether specialization improves harmful response detection.

Gemma-specific fine-tuning produced the strongest harmful-class F1-score among the model-specific DeBERTa variants, indicating that victim-model specialization can improve harmfulness detection depending on response style and refusal behavior.

### 5.5.5 PromptGuard Model-Specific Fine-tuning Results

To further analyze execution-aware prompt safety, PromptGuard was fine-tuned separately for each victim model: Gemma, LLaMA, and Qwen.

Instead of using a fully model-agnostic training set, each classifier was trained only on responses generated by its corresponding target model.

This allows the classifier to learn model-specific behavioral patterns and response styles, which often differ significantly across LLM families.

The results show clear variation across models.

### 5.5.6 Classifier $\times$ Victim Model Matrix

To summarize the final comparison, Table 5.11 presents classifier performance across model-agnostic and model-specific settings.

**Table 5.6** Model-Agnostic Classifier Performance

| Classifier | Accuracy | Macro F1 | Harmful Class F1 |
|---|---|---|---|
| DeBERTa | 0.8843 | 0.8676 | 0.8207 |
| PromptGuard (Balanced) | 0.8941 | 0.8791 | 0.8364 |
| PromptGuard (Full 9k) | 0.9554 | 0.8561 | 0.7365 |

**Table 5.7** DeBERTa Model-Agnostic Classification Report

| Class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| Benign (0) | 0.9377 | 0.8927 | 0.9146 | 354 |
| Harmful (1) | 0.7803 | 0.8654 | 0.8207 | 156 |
| Accuracy | 0.8843 | | | |
| Macro Avg | 0.8590 | 0.8790 | 0.8676 | 510 |
| Weighted Avg | 0.8896 | 0.8843 | 0.8859 | 510 |

**Table 5.8** DeBERTa Full 9k Classification Report

| Class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| Benign (0) | 0.9102 | 1.0000 | 0.9530 | 1693 |
| Harmful (1) | 0.0000 | 0.0000 | 0.0000 | 167 |
| Accuracy | 0.9102 | | | |
| Macro Avg | 0.4551 | 0.5000 | 0.4765 | 1860 |
| Weighted Avg | 0.8285 | 0.9102 | 0.8674 | 1860 |

**Table 5.9** DeBERTa Model-Specific Fine-tuning Results

| Victim Model | Accuracy | Macro F1 | Harmful Class F1 |
|---|---|---|---|
| Gemma-specific | 0.8170 | 0.7979 | 0.8600 |
| Llama-specific | 0.9304 | 0.7990 | 0.6364 |
| Qwen-specific | 0.7717 | 0.7711 | 0.7603 |

**Table 5.10** PromptGuard Model-Specific Fine-tuning Results

| Victim Model | Accuracy | Macro F1 | Harmful Class F1 |
|---|---|---|---|
| Gemma-specific | 0.8661 | 0.8160 | 0.7200 |
| Llama-specific | 0.9838 | 0.7590 | 0.5263 |
| Qwen-specific | 0.8849 | 0.6682 | 0.4000 |

### 5.5.7 Final Observation

Across all experiments, prompt-response based static classifiers consistently outperform prompt-only detection.

Balanced-dataset evaluation provides the most reliable estimate of safety performance under severe class imbalance.

PromptGuard achieves the strongest model-agnostic performance on the balanced dataset, with the highest harmful-class F1-score of 0.8364.

Model-specific fine-tuning improves harmful compliance detection for DeBERTa in certain victim models, particularly Gemma, where harmful-class F1 increases to 0.8600. However, PromptGuard performs best in the model-agnostic setting, and model-specific specialization leads to lower harmful-class performance across all three victim models.

These results show that response-aware classifiers are significantly more reliable than prompt-only filtering, and that the effectiveness of model specialization depends strongly on classifier architecture and victim-model behavior.

**Table 5.11** Final Classifier × Victim Model Matrix

| Classifier | Setting | Accuracy | Macro F1 | Harmful F1 |
|---|---|---|---|---|
| DeBERTa | Model Agnostic | 0.8843 | 0.8676 | 0.8207 |
| DeBERTa | Gemma-specific | 0.8170 | 0.7979 | 0.8600 |
| DeBERTa | Llama-specific | 0.9304 | 0.7990 | 0.6364 |
| DeBERTa | Qwen-specific | 0.7717 | 0.7711 | 0.7603 |
| PromptGuard | Model Agnostic (Balanced) | 0.8941 | 0.8791 | 0.8364 |
| PromptGuard | Gemma-specific | 0.8661 | 0.8160 | 0.7200 |
| PromptGuard | Llama-specific | 0.9838 | 0.7590 | 0.5263 |
| PromptGuard | Qwen-specific | 0.8849 | 0.6682 | 0.4000 |

# Chapter 6

# Discussion and Conclusion

## 6.1 Discussion

This thesis shows that prompt injection defence fails when harmfulness is treated as a property of prompt text alone.

The committee experiment produced heavy overlap between benign and malicious MPS distributions, with an AUC of only 0.5807. This confirms that prompt maliciousness is not a reliably separable text-level property. The same prompt may be safe for one model and harmful for another, depending on model behaviour and alignment.

This motivates execution-aware modelling, where safety is predicted from the joint prompt–response pair rather than from the prompt alone.

The execution-aware DeBERTa classifier outperformed prompt-only baselines and significantly improved harmful-class detection. The A4 multi-stage pipeline further improved robustness by combining a small analyzer with a large judge, achieving 99.17% accuracy and 100% true positive rate.

These results show that practical prompt-injection defence should move away from static prompt filtering and toward execution-aware, model-specific systems.

## 6.2 Limitations

This work has three main limitations.

- Large-scale labels were generated primarily using LLM judges. Although human validation showed 94.41% agreement, some bias may remain.
- Classifier training was evaluated on three open-weight models. Performance on larger proprietary models requires further study.
- The A4 pipeline requires multiple model calls, increasing inference cost for production deployment.

## 6.3 Conclusion

This thesis makes three core contributions.

1. It shows that prompt maliciousness is not an intrinsic property of text, and that prompt-only classification is insufficient for reliable defence.
2. It introduces execution-aware safety modelling using prompt–response pairs, where DeBERTa fine-tuning significantly improves harmful-class detection.
3. It demonstrates, through classifier $\times$ model evaluation and the A4 pipeline, that model-specific behaviour is central to practical prompt-injection defence.

Overall, effective LLM safety requires a shift from prompt-level filtering to execution-aware, model-specific defence systems.

# References


[Benjamin et al.(2024)] R. Benjamin et al. Systematically analyzing prompt injection vulnerabilities in diverse llm architectures. *arXiv preprint arXiv:2410.23308*, 2024.

[Chen et al.(2024)] Leo Chen et al. Spectral methods for prompt injection detection. *arXiv preprint arXiv:2404.00000*, 2024.

[Greshake et al.(2024)] Kai Greshake et al. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *USENIX Security Symposium*, 2024.

[Guo et al.(2024)] S. Guo et al. Sentinel: Multi-agent adversarial defense for llm safety. *arXiv preprint arXiv:2401.00000*, 2024.

[Jin et al.(2024)] Z. Jin et al. Large language model jailbreaking via adversarial rationales. *arXiv preprint arXiv:2402.10954*, 2024.

[Kang et al.(2024)] Minseok Kang et al. Covert prompt injection via markdown and formatting artifacts. In *IEEE Symposium on Security and Privacy*, 2024.

[Kim et al.(2024)] H. Kim et al. Variance risks in large language models: Instability as an indicator of model fragility. *International Journal of Neural Dynamics*, 5(1), 2024.

[Kwon et al.(2024)] Hyunwoo Kwon et al. Steganographic prompt injection attacks against large language models. volume 13, page 5008, 2024.

[Liu et al.(2024)] Xuan Liu et al. Automatic and universal prompt injection attacks against large language models. *arXiv preprint arXiv:2403.04957*, 2024.

[Liu et al.(2023)] Xuanwu Liu et al. Prompt injection attacks against llm-integrated applications. *arXiv preprint arXiv:2306.05499*, 2023.

[Ng et al.(2002)Ng, Jordan, and Weiss] Andrew Y. Ng, Michael Jordan, and Yair Weiss. On spectral clustering: Analysis and an algorithm. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2002.

[NVIDIA(2023)] NVIDIA. NeMo Guardrails. `https://github.com/NVIDIA/NeMo-Guardrails`, 2023.

[OpenAI(2024)] OpenAI. Model spec for safety-conscious ai assistants. OpenAI Technical Guidelines, 2024.

[Panda et al.(2023)] S. Panda et al. Why do models refuse to answer? aligning refusals via reasoning. *arXiv preprint arXiv:2310.00000*, 2023.

[Park et al.(2023)] J. Park et al. Generative agents for safe multi-agent llm systems. *arXiv preprint arXiv:2310.00000*, 2023.

[Perez et al.(2022)Perez, Ribeiro, et al.] Ethan Perez, Sonia Ribeiro, et al. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*, 2022.

[Wang et al.(2023)] L. Wang et al. Self-check: Using llms to zero-shot check their own step-by-step reasoning. *arXiv preprint arXiv:2308.13063*, 2023.

[Wei et al.(2023)] Jason Wei et al. Jailbreak llms via system prompt manipulation. *arXiv preprint arXiv:2307.02483*, 2023.

[Zeng et al.(2023)] J. Zeng et al. A comprehensive study of self-correction in large language models. *arXiv preprint arXiv:2311.00000*, 2023.

[Zou et al.(2023)] Andy Zou et al. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.